This architectural design focuses on **security, modularity, and high-performance data visualization**, which are critical for a Banking Management System.

---

1. UI Architecture

- **Framework:** Next.js 14+ (App Router) for SEO, server-side security, and edge-side rendering.
- **Language:** TypeScript (Strict mode).
- **Styling:** Tailwind CSS + Headless UI (for accessible primitives).
- **Communication:** TanStack Query (React Query) for server state; Axios for interceptor-based request handling (JWT management).

2. Folder Structure

```
src/
■■■ app/                  # App Router (Pages & Layouts)
■■■ components/           # Shared UI components
■   ■■■ ui/               # Atomic components (buttons, inputs)
■   ■■■ charts/           # Financial visualization (recharts)
■   ■■■ forms/            # Zod-validated form wrappers
■   ■■■ layout/           # Sidebar, Navbar, Breadcrumbs
■■■ features/             # Role-based business logic
■   ■■■ auth/
■   ■■■ transactions/
■   ■■■ loans/
■   ■■■ dashboard/
■■■ hooks/                # Global custom hooks (useUser, useToast)
■■■ lib/                  # Utilities (axios, auth config)
■■■ store/                # Zustand stores (Global UI state)
■■■ types/                # Shared domain types
```

3. Routing & Role-Based Access Control (RBAC)

We utilize a **Higher-Order Component / Middleware** approach:

- `/auth/*`: Public routes (Login, MFA, Reset Password).
- `/dashboard/customer/*`: Protected (Role: CUSTOMER).
- `/dashboard/employee/*`: Protected (Role: EMPLOYEE).
- `/dashboard/admin/*`: Protected (Role: ADMIN).

- **Middleware:** Checks for valid JWT/Session in every request; redirects to `/auth` if invalid.

4. State Management

- **Server State:** TanStack Query (caching, deduplication, background refetching).
- **Global UI State:** Zustand (lightweight, e.g., for Sidebar toggle, User session, Toast notifications).
- **Form State:** React Hook Form + Zod (for schema validation).

5. UI Libraries

- **Components:** [shadcn/ui](https://ui.shadcn.com/) (Highly customizable, accessible, built on Radix UI).
- **Icons:** Lucide React.
- **Charts:** Recharts (for financial trends and spending insights).
- **Table:** TanStack Table (server-side pagination, sorting, filtering).
- **Validation:** Zod.

6. Authentication Flow

1. **Login:** User enters credentials $\rightarrow$ API returns `auth_token` + `mfa_required_flag`.
2. **MFA Challenge:** Redirect to `/auth/mfa` $\rightarrow$ User submits OTP $\rightarrow$ API issues finalized `session_token`.
3. **Persistence:** JWT stored in an `httpOnly` secure cookie.
4. **Security:** Interceptors automatically attach `Authorization: Bearer ` to all requests.

7. Dashboard Layout Strategy

We use **Nested Layouts** in the Next.js App Router:

- `dashboard/layout.tsx`: Sidebar + Header + Main Content Area.
- The Sidebar changes dynamically based on the user's role (Fetched via `useAuth` hook).
- **Components included:**

* `Sidebar`: Collapsible, role-specific nav.

* `TransactionHistory`: Paginated list with filtering.

* `LoanWorkflow`: Kanban or Status-stepper board for Employees.

* `InsightCard`: AI-powered metric summaries.

8. Forms & Data Handling

- **Validation:** All forms must be defined via Zod schemas.
- **Example (Loan Application):**

```
const loanSchema = z.object({
  amount: z.number().min(1000).max(100000),
  reason: z.string().min(10),
  incomeProof: z.instanceof(File).refine(f => f.size < 5000000),
});
```

- **Interceptors:** Global Axios interceptor to handle `401 Unauthorized` (trigger logout) and `403 Forbidden` (redirect to error page).

9. Key Technical Features

- **Audit Logging:** Every user action triggered via custom hooks `useAudit(action, metadata)` which sends logs to a secure endpoint.
- **Performance:** React Query's `staleTime` and `cacheTime` configured to ensure the "under 2 seconds" requirement for transaction queries.
- **Accessibility:** WAI-ARIA roles throughout; all components must support keyboard navigation.
- **Security:** Masking of sensitive financial data by default; screen-reader-only labels for charts.

10. Next Steps for Implementation

1. **Auth Prototype:** Set up NextAuth.js or custom JWT auth with MFA.
2. **Mock API:** Use MSW (Mock Service Worker) to simulate transactions and loan workflows.
3. **UI Shell:** Build the sidebar and layout wrappers with `shadcn/ui`.
4. **Feature Sprints:** Build "Transactions" (Customer) and "Loan Approval" (Employee) modules using TanStack Query.